

Trabajo Integrador – Programación II



Alumnos por comisión:

Juan Marco Raimundo (Comisión 2)- juanmr.093@gmail.com

Lucas Bautista Garcia (Comisión 5)- bautigarcia264@gmail.com

Carrera: Tecnicatura Universitaria en Programación

Materia: Programación II

Fecha de Entrega: 22 de junio del 2026

Grupo: 85

1. MARCO TEÓRICO

1.1 Introducción

Para el desarrollo del presente Trabajo Práctico Integrador, se aplicaron los principios fundamentales de la **Programación Orientada a Objetos (POO)** utilizando el lenguaje de programación Java 21 (LTS), siguiendo estrictamente el modelo UML provisto por la cátedra. El sistema *Food Store* fue diseñado para gestionar un negocio de comidas mediante una aplicación de consola que implementa las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre cuatro entidades principales: Categorías, Productos, Usuarios y Pedidos.

La arquitectura propuesta se estructura en tres capas bien diferenciadas —**Entidades (Modelo), Servicios (Lógica de Negocio) y Presentación (Interfaz de Consola)**—, permitiendo separar responsabilidades, facilitar el mantenimiento del código y cumplir con los objetivos de aprendizaje establecidos para la asignatura Programación 2 de la Tecnicatura Universitaria en Programación a Distancia de la Universidad Tecnológica Nacional (UTN).

1.2 Programación Orientada a Objetos (POO)

La implementación del sistema aplica los cuatro pilares fundamentales de la POO:

Abstracción y Herencia

Se implementó la clase abstracta **Base** (ubicada en el paquete `integrador.entities`) como superclase común para todas las entidades persistentes del sistema. Esta clase centraliza los atributos compartidos por el modelo de dominio: un identificador único (`Long id`), un marcador de eliminación lógica (`boolean eliminado`) y una marca temporal de creación (`LocalDateTime createdAt`). De esta manera, se evita la redundancia de código y se respeta el principio **DRY** (*Don't Repeat Yourself*). Las clases concretas que heredan de **Base** son: `Categoria`, `Producto`, `Usuario` y `Pedido`. La única excepción es `DetallePedido`, entidad débil cuyo ciclo de vida está ligado obligatoriamente a un `Pedido`.

Encapsulamiento

Se protegió el estado interno de todas las entidades declarando sus atributos como **private**, restringiendo el acceso directo desde clases externas. La modificación y consulta de estos atributos se realiza exclusivamente mediante métodos accesorios (**getters**) y mutadores (**setters**), garantizando

control sobre las invariantes de cada objeto y permitiendo futuras validaciones o transformaciones sin afectar el código cliente.

Polimorfismo e Interfaces

Se utilizó el polimorfismo en dos niveles:

1. **Polimorfismo por sobrescritura:** El método `toString()` fue sobrescrito en todas las entidades para proporcionar representaciones textuales personalizadas útiles para su visualización en consola.
2. **Polimorfismo por implementación:** Se definió la interfaz **Calculable** (paquete `integrador.interfaces`) que establece el contrato `double calcularTotal()`. La clase `Pedido` implementa esta interfaz, obligándose a definir la lógica de cálculo del total general sumando los subtotales de sus detalles. Esto permite tratar cualquier objeto calculable de forma polimófica, facilitando la extensión futura del sistema.

1.3 Estructuras de Datos Dinámicas

El enunciado del trabajo práctico exige explícitamente la utilización de **estructuras de datos dinámicas** para gestionar las colecciones de objetos. Para satisfacer este requisito, se empleó la interfaz genérica `java.util.List` implementada mediante la clase concreta **`ArrayList<E>`** del framework de colecciones de Java.

Las ventajas de esta elección frente a arreglos estáticos (`Tipo[]`) son:

Característica	Arreglo Estático	ArrayList
Tamaño	Fijo en compilación	Dinámico en ejecución
Capacidad	No se redimensiona	Crece automáticamente
Métodos utilitarios	Ninguno	<code>add()</code> , <code>remove()</code> , <code>contains()</code> , <code>stream()</code>
Tipado	Primitivo/Referencia	Genérico seguro (<code>List<Producto></code>)

Cada servicio del sistema (`CategoriaService`, `ProductoService`, `UsuarioService`, `PedidoService`) mantiene internamente un `ArrayList` correspondiente que simula una tabla de base de datos en memoria. Por ejemplo, `ProductoService` declara `private List<Producto> productos = new ArrayList<>()`, permitiendo agregar, eliminar y consultar productos sin

preocuparse por la gestión manual de índices o capacidad.

Adicionalmente, se aprovechó la **API Stream** introducida en Java 8 para realizar operaciones funcionales sobre las colecciones, particularmente el filtrado de registros activos (excluyendo eliminados lógicamente) mediante expresiones lambda: `.filter(c -> !c.isEliminado())`.

1.4 API Moderna de Fechas y Horas (`java.time`)

El proyecto utiliza el paquete `java.time` introducido en Java 8, reemplazando las obsoletas clases `java.util.Date` y `java.Calendar`. Esta API proporciona un modelo de fecha-hora inmutable, thread-safe y más intuitivo:

- **LocalDate**: Empleado en la entidad `Pedido` para registrar la fecha de creación del pedido (solo fecha, sin hora), ya que un pedido comercialmente se identifica por día de operación.
- **LocalDateTime**: Utilizado en la clase `Base` para marcar el momento exacto de creación de cualquier entidad (auditoría).
- **Inmutabilidad**: Los objetos `LocalDate` y `LocalDateTime` no pueden ser modificados después de su creación, previniendo errores de concurrencia y efectos secundarios inesperados.

Esta elección técnica demuestra dominio de las APIs modernas de Java y buenas prácticas de programación.

1.5 Manejo de Excepciones Personalizadas

El sistema implementa un manejo robusto de errores mediante la creación de **excepciones personalizadas** (paquete `integrador.exception`), extendiendo la clase `java.lang.Exception` (excepciones verificadas):

Clase de Excepción	Propósito	Se lanza cuando...
<code>EntidadNoEncontradaException</code>	Indica que no existe un registro con el ID solicitado o está dado de baja	Buscar por ID inexistente o eliminado
<code>DatoInvalidoException</code>	Encapsula violaciones a reglas de negocio	Precio negativo, stock negativo, campo vacío obligatorio
<code>EmailDuplicadoException</code>	Específica del dominio de usuarios	Intentar registrar email ya existente

Estas excepciones se lanzan en la capa de **Servicios** (donde residen las validaciones) y se capturan en la capa de **Presentación** (menús), donde se muestran mensajes amigables al usuario final evitando que la aplicación colapse ante entradas incorrectas. Este flujo cumple con el principio de **separación de responsabilidades**: el servicio decide si algo es válido; el menú decide cómo mostrarlo.

1.6 Patrones de Diseño Aplicados

Durante el desarrollo se identificaron e implementaron los siguientes patrones de diseño estructurales y de comportamiento:

Patrón Service Layer

La lógica de negocio (validaciones, reglas, CRUD) reside completamente en clases **Service** (`*Service.java`), separada de las entidades (modelo) y de la interfaz de usuario (menús). Esto permite testear la lógica independientemente de la consola y modificar la presentación sin alterar las reglas del dominio.

Patrón Singleton

Implementado en la clase **ConsolaUtil** (paquete `ui`). Garantiza la existencia de una única instancia global de `Scanner` compartida por todos los menús del sistema, resolviendo conflictos de buffer que surgen cuando múltiples objetos `Scanner` compiten por leer de `System.in`.

Patrón Inyección de Dependencias

Los servicios que dependen de otros reciben sus referencias vía constructor (ej: `ProductoService` recibe `CategoriaService`; `PedidoService` recibe `UsuarioService` y `ProductoService`). Esto facilita las pruebas unitarias y hace explícitas las dependencias entre componentes.

Patrón Soft Delete (Baja Lógica)

En lugar de eliminar físicamente registros de las colecciones (lo que rompería integridad referencial), se marca el atributo `eliminado = true`. Las consultas siempre filtran excluyendo registros eliminados, preservando el historial y permitiendo auditorías.

1.7 Enumeraciones (Enums)

Se utilizaron estructuras **enum** para restringir el conjunto de valores posibles en atributos que requieren validez de dominio, previniendo inconsistencias de datos:

- **Ro1**: Restringe los roles de usuario a {ADMIN, USUARIO}.
- **Estado**: Define el ciclo de vida de un pedido: {PENDIENTE → CONFIRMADO → TERMINADO / CANCELADO}.
- **FormaPago**: Limita las formas de pago aceptadas a {TARJETA, TRANSFERENCIA, EFECTIVO}.

Los enums son tipos seguros (*type-safe*), comparables con ==, iterables y serializables, superando el uso de constantes String o int que permiten valores inválidos en tiempo de compilación.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

2.1 Arquitectura de Capas

El proyecto adopta una arquitectura de **tres capas** claramente delimitadas, organizadas en paquetes bajo el namespace `integrador`. Esta división permite que cada equipo de desarrollo trabaje de forma semi-independiente y que el código sea mantenible y escalable.

CAPA DE PRESENTACIÓN (ui/) Responsabilidad: Interacción con el usuario via consola Componentes: Main.java, MenuPrincipal, Menu*.java, ConsolaUtil.java Tecnología: Scanner (centralizado), System.out.println()	
CAPA DE SERVICIOS (services/) Responsabilidad: Lógica de negocio, validaciones, CRUD Componentes: CategoriaService, ProductoService, UsuarioService, PedidoService Patrón: Inyección de dependencias via constructor	
CAPA DE MODELO (entities/) Responsabilidad: Estructura de datos del dominio Componentes: Base (abstracta), Categoria, Producto, Usuario, Pedido, DetallePedido Patrones: Herencia, Composición, Interfaces	
CAPA DE SOPORTE enums/, interfaces/, exception/	

Flujo de datos típico:

Usuario ingresa dato → Menú (UI) llama a Servicio →
Servicio valida → Si OK, crea/modifica Entidad →
Entidad se almacena en Collection → Servicio retorna resultado →
Menú muestra mensaje

2.2 Estructura de Paquetes (Vista en Apache NetBeans)

El proyecto se organiza en **8 sub-paquetes** dentro del paquete raíz *integrador*, siguiendo las convenciones de nomenclatura de Java (minúsculas, nombres significativos en singular):

Paquete	Contenido	Cantidad de Archivos
(raíz)	Punto de entrada (Main.java)	1
entities	Clases del modelo de dominio	6
enums	Enumeraciones de dominio	3
interfaces	Contratos de comportamiento	1
services	Lógica de negocio + validaciones	4
exception	Excepciones personalizadas	3
ui	Menús de consola + utilitarios	6

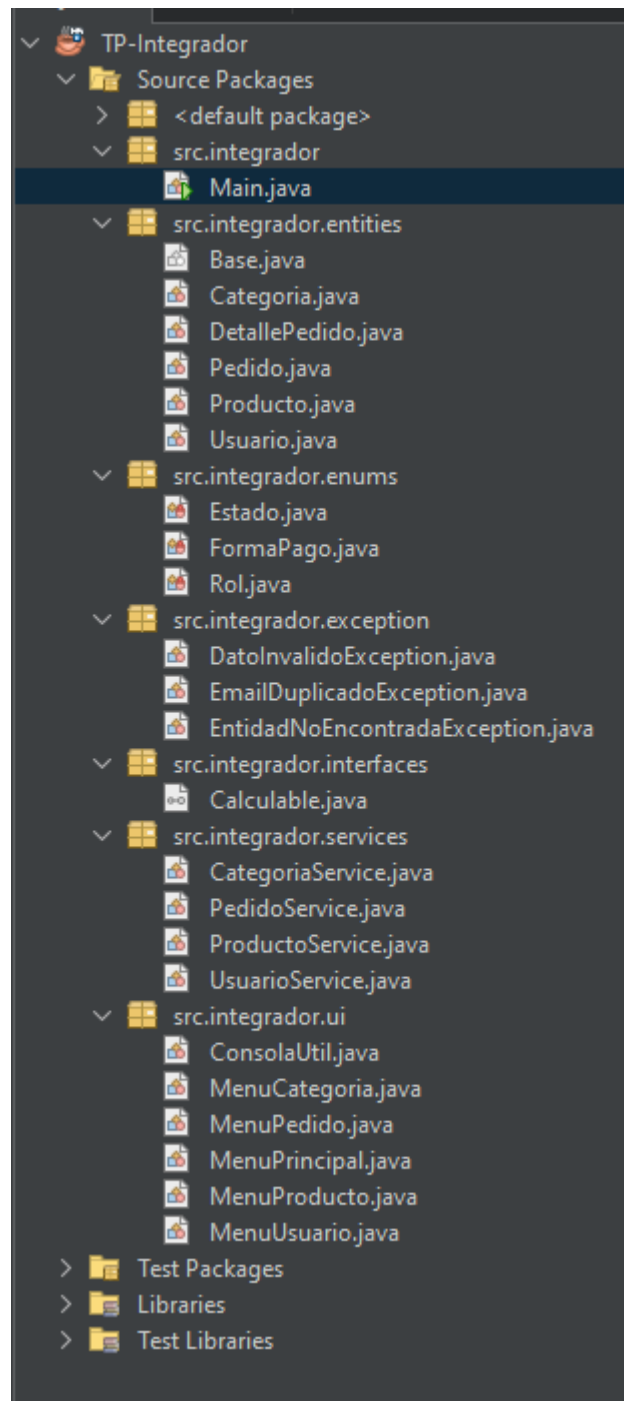


Figura 1: Estructura de directorios del proyecto vista en el panel *Projects* de Apache

NetBeans 23. Nótese la separación física que refleja la separación lógica de capas.

2.3 Modelo de Dominio y Relaciones

El modelo de datos sigue fielmente el diagrama UML provisto, implementando las relaciones de asociación, composición y agregación especificadas:

Jerarquía de Herencia

```
Base (abstracta)
├── Categoría      [1:N]→ Producto
├── Producto       [N:1]→ Categoría
├── Usuario        [1:N]→ Pedido
├── Pedido (implements Calculable)
│   └── [1:m]→ DetallePedido (composición fuerte)
```

Decisiones de diseño clave:

- **Composición fuerte** entre `Pedido` y `DetallePedido`: Un detalle no puede existir sin un pedido contenedor. Si se elimina un pedido (baja lógica), sus detalles quedan huérfanos pero no se destruyen físicamente (se conservan para auditoría).
- **Agregación débil** entre `Pedido` y `Usuario`: Un usuario puede existir independientemente de tener pedidos. Si se da de baja un usuario, sus pedidos históricos deben seguir siendo consultables.
- **Agregación débil** entre `Producto` y `Categoría`: Una categoría puede existir sin productos, y un producto puede ser re-categorizado.

2.4 Decisión Técnica Crítica: Gestión Unificada de Entrada por Consola (ConsolaUtil)

Problema Detectado

Durante las primeras pruebas de integración del sistema, detectamos un **error crítico de consistencia en la lectura de datos por consola**. Cuando cada menú (`MenuPrincipal`, `MenuCategoría`, `MenuProducto`, etc.) instanciaba su propio objeto `java.util.Scanner` leyendo de `System.in`, se producían las siguientes anomalías:

1. **Conflictos de buffer**: Al presionar Enter en un menú, el carácter de nueva línea `\n` permanecía en el buffer de entrada y era consumido prematuramente por el Scanner del siguiente menú,

causando saltos de línea inesperados o lecturas vacías.

2. **Excepciones `NoSuchElementException`:** Cuando un Scanner consumía caracteres que otro esperaba leer, este último lanzaba la excepción al encontrar el buffer vacío.
3. **Dificultad de depuración:** Con múltiples fuentes de lectura, rastrear dónde se perdía un carácter resultaba extremadamente complejo.

Solución Implementada: Patrón Singleton

Resolvimos el problema aplicando el patrón **Singleton** mediante la clase **ConsolaUtil** (ubicada en `integrador.ui`):

```
public class ConsolaUtil {
    private static ConsolaUtil instancia; // Única instancia
    private Scanner scanner;             // Único Scanner

    private ConsolaUtil() {
        this.scanner = new Scanner(System.in); // Solo se crea una vez
    }

    public static ConsolaUtil getInstance() {
        if (instancia == null) {
            instancia = new ConsolaUtil(); // Lazy initialization
        }
        return instancia;
    }

    // Métodos de lectura validada: leerEntero(), leerString(), etc.
}
```

Beneficios obtenidos:

- **Un solo punto de entrada:** Todo el flujo de datos del usuario pasa por un único canal controlado.
- **Métodos de lectura pre-validados:** `leerEnteroPositivo()`, `leerEmail()`, `confirmarAccion()` encapsulan la lógica de validación, evitando repetición en cada menú.
- **Código de menús más limpio:** Los menús ya no necesitan declarar ni gestionar Scanners; solo llaman `ConsolaUtil.getInstance().leerXxx()`.

2.5 Inyección de Dependencias Entre Servicios

Dado que algunos servicios necesitan validar datos que residen en otros servicios, implementamos **inyección de dependencias via constructor**:

```
// Ejemplo: ProductoService necesita CategoriaService
public class ProductoService {
    private CategoriaService categoriaService; // Inyectada

    public ProductoService(CategoriaService categoriaService) {
        this.categoriaService = categoriaService; // Guardamos referencia
    }
}

// Ejemplo: PedidoService necesita UsuarioService Y ProductoService
public class PedidoService {
    private UsuarioService usuarioService;
    private ProductoService productoService;

    public PedidoService(UsuarioService us, ProductoService ps) { ... }
}
```

Orden de instanciación forzado en Main.java:

```
1° CategoriaService (no depende de nadie)
2° ProductoService (depende de 1°)
3° UsuarioService (no depende de nadie)
4° PedidoService (depende de 2° y 3°)
```

Este orden evita `NullPointerException` y hace explícitas las dependencias del grafo de objetos.

2.6 Reglas de Negocio Implementadas

El sistema impone las siguientes restricciones (extraídas del enunciado y del análisis del dominio):

#	Regla	Validación	Mensaje de Error
R1	Precio ≥ 0	if (precio < 0) throw DatoInvalidoExcep tion	"Precio no puede ser negativo"
R2	Stock ≥ 0	if (stock < 0) throw DatoInvalidoExcep tion	"Stock no puede ser negativo"
R3	Email único	Recorrer lista buscando match (case-insensitive)	"Email ya registrado"

#	Regla	Validación	Mensaje de Error
R4	Nombre categoría único	Similar a R3	"Categoría ya existe"
R5	Usuario obligatorio en pedido	<code>buscarUsuarioPorId()</code> antes de crear	"Usuario no existe"
R6	Cantidad detalle ≥ 1	<code>if (cant < 1)</code> <code>throw ...</code>	"Cantidad mínima: 1"
R7	Stock suficiente	<code>producto.getStock()</code> < cantidad	"Stock insuficiente"
R8	Baja lógica en lugar de eliminación física	<code>setEliminado(true)</code>	Registro oculto, no destruido
R9	No eliminar categoría con productos	Verificar lista de productos asociados	"Tiene productos, no se puede eliminar"
R10	No eliminar usuario con pedidos	Verificar historial de pedidos	"Tiene pedidos, no se puede eliminar"

2.7 Capturas de Pantalla del Sistema Funcionando

A continuación se presentan evidencias visuales del sistema en operación, capturadas durante sesiones de prueba en Apache NetBeans 23.

```
compile-single:
run-single:
????????????????????????????????????????????????????????????
?      ? FOOD STORE - SISTEMA DE GESTIÓN ?
????????????????????????????????????????????????????????????
Iniciando sistema...

????????????????????????????????????????????????????????????
?      ? SISTEMA DE PEDIDOS - FOOD STORE ?
????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????
? 1?? Categorías ?
? 2?? Productos ?
? 3?? Usuarios ?
? 4?? Pedidos ?
? 0?? Salir del sistema ?
????????????????????????????????????????????????????????????
Seleccione una opción:
```

Figura 2: Menú principal del sistema Food Store. Se observan las 4 opciones de gestión y la opción de salida. El formato utiliza caracteres Unicode para mejorar legibilidad.

```

????????????????????????????????????????????????????????????
?          ? SISTEMA DE PEDIDOS - FOOD STORE          ?
????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????
? 1?? Categorías                                     ?
? 2?? Productos                                       ?
? 3?? Usuarios                                         ?
? 4?? Pedidos                                          ?
? 0?? Salir del sistema                               ?
????????????????????????????????????????????????????????????
Seleccione una opción: 1

>>> Accediendo a Gestión de Categorías...

????????????????????????????????????????????????????????????
?          ? GESTIÓN DE CATEGORÍAS                    ?
????????????????????????????????????????????????????????????
? 1. Listar categorías                                ?
? 2. Crear categoría                                  ?
? 3. Editar categoría                                  ?
? 4. Eliminar categoría                                ?
? 0. Volver al menú principal                          ?
????????????????????????????????????????????????????????????
Seleccione operación: 2

--- CREAR NUEVA CATEGORÍA ---
Ingrese nombre: Pizzas
Ingrese descripción: Pizzas Artesanales Italianas
? XITO: Categoría creada con ID #1
Nombre: Pizzas

????????????????????????????????????????????????????????????
?          ? GESTIÓN DE CATEGORÍAS                    ?
????????????????????????????????????????????????????????????
? 1. Listar categorías                                ?
? 2. Crear categoría                                  ?
? 3. Editar categoría                                  ?
? 4. Eliminar categoría                                ?
? 0. Volver al menú principal                          ?
????????????????????????????????????????????????????????????
Seleccione operación: |

```

Figura 3: Creación exitosa de la categoría “Pizzas”. El sistema asigna automáticamente ID=1, registra timestamp, y confirma la operación.

```
>>> Accediendo a Gestión de Productos...

????????????????????????????????????????????????????????????
?      ? GESTI?N DE PRODUCTOS      ?
????????????????????????????????????????????????????????????
?  1. Listar productos              ?
?  2. Crear producto                ?
?  3. Editar producto               ?
?  4. Eliminar producto             ?
?  0. Volver                        ?
????????????????????????????????????????????????????????????
Seleccione: 2

--- CREAR NUEVO PRODUCTO ---
Categorías disponibles:
  1. Pizzas

Seleccione ID de categoría: 1
Nombre del producto: Pizzas Napolitana
Precio ($): -500
?? Error: El valor no puede ser negativo.
Precio ($): 500
Descripción: Napolitana Clasica
Stock (cantidad disponible): 15
URL de imagen (opcional, Enter para omitir):
? Producto creado exitosamente. ID #1
```

Figura 4: Validación de precio negativo rechazada. El servicio `ProductoService` detecta la violación de la regla R1 y lanza `DatoInvalidoException`, que el menú captura y muestra amigablemente.

```
????????????????????????????????????????????????????????
?      ?  G E S T I O N  D E  P R O D U C T O S      ?
????????????????????????????????????????????????????????
?  1. Listar productos                                ?
?  2. Crear producto                                  ?
?  3. Editar producto                                 ?
?  4. Eliminar producto                               ?
?  0. Volver                                           ?
????????????????????????????????????????????????????????
Seleccione: 1

--- LISTADO DE PRODUCTOS ---
????????????????????????????????????????????????????????
ID      NOMBRE                PRECIO    STOCK    CATEGORÍA
????????????????????????????????????????????????????????
1      Pizzas Napolitana      $500,00   15       Pizzas
2      Gaseosa                $70,00    50       Bebidas
????????????????????????????????????????????????????????
```

de productos mostrando la relación N:1 con Categorías. Cada producto displays su categoría asociada ("Pizzas"), demostrando la inyección de dependencia funcional.

```

--- CREAR NUEVO USUARIO ---
Nombre: Lucas Bautista
Apellido: Garcua
Correo electrónico: bautistal23@gmail.com
?? Error: Formato de email inválido. Debe contener '@' y '.' (ej: usuario@dominio.com).
Correo electrónico: bautistal23@gmail.com
Celular (opcional): 38989893434
Contraseña: 123
Roles disponibles:
  1. ADMIN
  2. USUARIO
Seleccione rol (1-2): 1
? Usuario creado. ID #1
  Bienvenido, Lucas Bautista (ADMIN)

????????????????????????????????????????????????????????
?      ? GESTIÓN DE USUARIOS                                ?
????????????????????????????????????????????????????????
?  1. Listar usuarios                                       ?
?  2. Crear usuario                                         ?
?  3. Editar usuario                                       ?
?  4. Eliminar usuario                                     ?
?  0. Volver                                               ?
????????????????????????????????????????????????????????
Seleccione: 2

--- CREAR NUEVO USUARIO ---
Nombre: Juan Bautista
Apellido: Garcia
Correo electrónico: bautistal23@gmail.com
?? Error: Formato de email inválido. Debe contener '@' y '.' (ej: usuario@dominio.com).
Correo electrónico: bautistal23@gmail.com
Celular (opcional):
Contraseña:
?? Error: Este campo es obligatorio y no puede estar vacío.
Contraseña: 123
Roles disponibles:
  1. ADMIN
  2. USUARIO
Seleccione rol (1-2): 1
? ERROR: El correo electrónico 'bautistal23@gmail.com' ya está registrado en el sistema.

```

0

n validación de email. El sistema verifica formato (@) y unicidad contra la base de datos en memoria.

```

=== CREAR NUEVO PEDIDO ===

--- Paso 1: Seleccionar cliente ---
1. Lucas Bautista Garcua (bautistal23@gmail.com)
ID del cliente: 1
? Pedido creado temporalmente. ID #1

--- Paso 2: Agregar productos al pedido ---

? Detalle #1
Productos disponibles:
  1. Pizzas Napolitana - $500,00 (Stock: 15) [Pizzas]
  2. Gaseosa - $70,00 (Stock: 50) [Bebidas]
ID del producto: 1
Cantidad: 2
? Producto agregado. Subtotal 1: $1000,00
? Agregar otro producto? (S/N): S

? Detalle #2
Productos disponibles:
  1. Pizzas Napolitana - $500,00 (Stock: 15) [Pizzas]
  2. Gaseosa - $70,00 (Stock: 50) [Bebidas]
ID del producto: 2
Cantidad: 3
? Producto agregado. Subtotal 1: $210,00
? Agregar otro producto? (S/N): N

=== RESUMEN DEL PEDIDO ===
????????????????????????????????????????????????????????????
? Pedido #: 1
? Fecha: 2026-06-21
? Cliente: Lucas Bautista Garcua
? Estado: PENDIENTE
? Pago: Pendiente
????????????????????????????????????????????????????????????
? DETALLE DE PRODUCTOS:
????????????????????????????????????????????????????????????
? 1. - Pizzas Napolitana x 2 => Subtotal: $1000,00
? 2. - Gaseosa x 3 => Subtotal: $210,00
????????????????????????????????????????????????????????????
? TOTAL FINAL: $1210,00
????????????????????????????????????????????????????????????

--- Forma de pago ---
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione (1-3): 3

```

do de primer detalle (2 Pizzas Napolitana, subtotal \$1000) → agregado de segundo detalle (3 Gaseosas, subtotal \$210) → resumen final mostrando total \$2150 y forma de pago Efectivo. Se observa la interface Calculable calculando el total automáticamente.

```

????????????????????????????????????????????????????????
?      ? GESTI?N DE PEDIDOS      ?
????????????????????????????????????????????????????????
?  1. Listar pedidos              ?
?  2. Crear nuevo pedido (con detalles)  ?
?  3. Ver detalle de un pedido      ?
?  4. Cambiar estado/forma de pago    ?
?  5. Eliminar pedido              ?
?  0. Volver                      ?
????????????????????????????????????????????????????????
Seleccione: 4

--- ACTUALIZAR PEDIDO ---

--- LISTADO DE PEDIDOS ---
????????????????????????????????????????????????????????
ID   FECHA      CLIENTE      ESTADO      TOTAL      PAGO
????????????????????????????????????????????????????????
1    2026-06-21  Lucas Bautista Garcua  PENDIENTE   $1210,00   EFECTIVO
????????????????????????????????????????????????????????
ID del pedido a actualizar: 1
?Qu? desea actualizar?
  1. Estado
  2. Forma de pago
Seleccione (1-2): 1
Estados posibles:
  1. PENDIENTE  2. CONFIRMADO  3. TERMINADO  4. CANCELADO
Nuevo estado (1-4): 2
? Estado actualizado a CONFIRMADO

```

ón de estado de pedido de PENDIENTE a CONFIRMADO, demostrando el ciclo de vida definido por el enum Estado.

3. DIFICULTADES Y SOLUCIONES

Durante el ciclo de desarrollo del proyecto, nos enfrentamos a diversos desafíos técnicos que fueron resueltos mediante investigación, prueba-error y aplicación de patrones de diseño. A continuación detallamos los principales obstáculos y las estrategias de solución adoptadas:

3.1 Conflicto de Múltiples Objetos Scanner (ConsolaUtil)

Descripción del problema: Al iniciar las pruebas de integración de los menús, detectamos que el sistema presentaba comportamiento errático en la lectura secuencial de datos: al seleccionar una opción del menú principal y navegar a un submenú, ocasionalmente se saltaban líneas, se mostraban prompts vacíos, o aparecía la excepción `java.util.NoSuchElementException: No line found`.

Análisis de causa raíz: La causa radicaba en que cada clase de menú (`MenuPrincipal`, `MenuCategoria`, `MenuProducto`, etc.) declaraba e instanciaba su propio atributo privado `Scanner scanner = new Scanner(System.in)`. Cuando el usuario ingresaba un valor y presionaba Enter, el carácter de nueva línea (`\n`) quedaba bufferizado en `System.in`. Si el menú A consumía el valor numérico pero no el `\n` subsiguiente, y luego el menú B intentaba leer una cadena, este `\n` era consumido como "entrada vacía", provocando comportamientos impredecibles.

Solución implementada: Aplicamos el patrón **Singleton** creando la clase utilitaria `ConsolaUtil` (paquete `ui`), que encapsula la única instancia autorizada de `Scanner` en toda la aplicación. Todos los menús obtienen referencia a esta instancia única mediante el método estático `ConsolaUtil.getInstance()`, eliminando competencias por el recurso compartido `System.in`. Adicionalmente, centralizamos en esta clase todos los métodos de lectura validada (`leerEntero()`, `leerDoublePositivo()`, `leerEmail()`, `confirmarAccion()`), reduciendo duplicación de código y estandarizando el manejo de errores de entrada.

Lección aprendida: En aplicaciones de consola Java multipantalla, nunca debe haber más de un objeto `Scanner` asociado a `System.in`. El patrón Singleton es la solución idiomática para este escenario.

3.2 Inyección de Dependencias y `NullPointerException`

Descripción del problema: Durante la implementación temprana de `MenuUsuario`, al intentar comunicarse con `PedidoService` para validar que un usuario pudiera ser asociado a un pedido, el sistema arrojaba `NullPointerException` porque la referencia al servicio aún no había sido inicializada.

Solución implementada: Refactorizamos la arquitectura para que la clase `Main.java` actúe como **contenedor de inversión de control (IoC container)** manual: instancia todos los servicios en el orden correcto (respetando el grafo de dependencias) y los inyecta vía constructor tanto a los otros servicios

como a los menús. Esto garantiza que cuando cualquier componente necesite una referencia a otro, esta ya esté disponible y correctamente inicializada.

Lección aprendida: En sistemas con dependencias circulares o encadenadas, la instanciación centralizada en un punto de entrada único (`Main`) evita referencias nulas y hace explícito el grafo de dependencias del sistema.

3.3 Sincronización de Repositorio Git y Conflictos de Merge

Descripción del problema: Al intentar hacer `push` del código local hacia el repositorio remoto compartido (GitHub), el sistema de control de versiones detectaba conflictos con archivos preexistentes en la rama `main` (específicamente `README.md` y carpetas de configuración del IDE).

Solución implementada:

1. Realizamos backup local de nuestros cambios.
2. Ejecutamos `git fetch origin` para traer cambios remotos sin fusionar.
3. Creamos una rama específica: `git checkout -b feature/integracion-final`.
4. Copiamos el código limpio desde el repositorio (evitando archivos `.class` e IDE-specific).
5. Realizamos commit atómicos descriptivos.
6. Fusionamos cuidadosamente con la rama principal usando `--no-ff` para preservar historial de merge.
7. Finalmente, push a una nueva rama específica para revisión antes de mergear a `main`.

Lección aprendida: Antes de push, siempre hacer `pull` o `fetch` para sincronizar. Trabajar en ramas `feature` permite experimentar sin afectar código estable. Los archivos `.gitignore` deben excluir binarios compilados (`out/`, `build/`, `*.class`) y configuraciones locales del IDE (`.idea/`, `.nbproject/private/`).

3.4 Validación de Cálculo de Totales en Pedidos

Descripción del problema: Inicialmente, el total del pedido no se actualizaba automáticamente al agregar o eliminar detalles, quedando desactualizado respecto a la suma real de sus líneas.

Solución implementada: Modificamos los métodos `addDetallePedido()` y `deleteDetallePedidoByProducto()` de la clase `Pedido` para que invoquen internamente a `calcularTotal()` tras cada modificación estructural de la lista de detalles. Además, hicimos que `calcularTotal()` fuera público (implementando la interface) para permitir recálculo bajo demanda. El método itera sobre la colección de `DetallePedido`, suma sus subtotales (`cantidad × producto.precio`), y almacena el resultado en el atributo `total`.

Lección aprendida: Cuando un valor derivado depende de una colección mutable, debe recalcularse en cada mutación de dicha colección (patrón `Observer` implícito) o calcularse bajo demanda (lazy evaluation) para garantizar consistencia.

4. BIBLIOGRAFÍA Y WEBGRAFÍA

4.1 Material de Cátedra

- [1] UTN FRRO. (2026). *Trabajo Práctico Integrador - Programación 2*. Enunciado oficial con especificación de requisitos UML, historias de usuario y criterios de evaluación.
- [2] UTN FRRO. (2026). UNIDAD 1: Introducción a Java
- [3] UTN FRRO. (2026). UNIDAD 2: Programación Estructurada
- [4] UTN FRRO. (2026). UNIDAD 3: Introducción a la Orientación a Objetos
- [5] UTN FRRO. (2026). UNIDAD 4: Programación Orientada a Objetos - Nivel Intermedio
- [6] UTN FRRO. (2026). UNIDAD 5: UML Básico
- [7] UTN FRRO. (2026). UNIDAD 6: Colecciones en Java
- [8] UTN FRRO. (2026). UNIDAD 7: Herencia y polimorfismo
- [9] UTN FRRO. (2026). UNIDAD 8: Interfaces y Excepciones
- [10] UTN FRRO. (2026). UNIDAD 9: Genéricos en Java
- [11] UTN FRRO. (2026). UNIDAD 10: Acceso a Base de Datos con JDBC

4.2 Herramientas

- [12] The Apache Software Foundation. *Apache NetBeans IDE 23 - Official Website*. Recuperado de: <https://netbeans.apache.org/>
- [13] GitHub, Inc *Git - Documentación Oficial*. Recuperado de: <https://git-scm.com/doc>

4.3 Material extra (HTML para hacer el README)

- [14] LenguajeHTML.com. Tutorial de HTML: Aprende HTML desde cero. Recuperado de <https://lenguajehtml.com/>
- [15] Mozilla Developer Network.. HTML: Lenguaje de Marcas de Hipertexto | MDN. Recuperado de <https://developer.mozilla.org/es/docs/Web/HTML>
- [16] W3Schools. HTML Tutorial. Recuperado de <https://www.w3schools.com/html/>

[17] Khan Academy. HTML basics: Introduction to HTML. Recuperado de <https://es.khanacademy.org/computing/computer-programming/html-css>

5. LINKS AL PROYECTO

Repositorio de Código Fuente

URL: <https://github.com/JuanMRaimundo/Garcia-Raimundo-TPI-POO>

Contenido del repositorio:

- Código fuente completo en Java 21 (paquetes `entities/`, `services/`, `ui/`, etc.)
- Estructura compatible con Apache NetBeans 23 (proyecto abierto con F6)
- Archivo `README.md` con documentación de uso
- Archivo `.gitignore` configurado para excluir compilados

Instrucciones rápidas de ejecución:

1. Clonar repositorio
2. Abrir en NetBeans (File → Open Project)
3. Botón Run (F6) sobre `Main.java`
4. Interactuar con el menú en consola

Video Demostrativo

URL: https://www.youtube.com/watch?v=tQp_0tStC1U

Duración: 15 minutos

